

Lesson 5 Teacher's Script

Programming Basics IV: Enemy Ranks

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week you got a working enemy - shooting it destroyed it cleanly, even with a few misses mixed in. Quick show of hands - who got that working? Anyone try the Level 3 ideas?" [Pause, call on 1-2 volunteers]

"Today, one enemy becomes many. By the end of today, your game will have a whole row of enemies, and you'll do it without writing the same setup code five separate times."

[0:05-0:25] Loops: Doing Something Many Times Without Repeating Yourself

"Quick question: if I wanted five enemies on screen right now, what's the laziest way? " [Pause for guesses - someone will likely say "copy and paste the code five times"] "That would actually work! But imagine changing something about how enemies look later - now you're editing five separate copies instead of one. That's exactly the problem a loop solves."

"A loop just means: run this same block of instructions a certain number of times, instead of writing it out repeatedly. You've already used a version of this without maybe noticing - your bullets loop and your movement checks run every single frame, which is itself a loop the engine drives for you. Today we write our own loop, on purpose, to create several enemies from one block of code."

[Demo cue: switch to GameMaker, open a project with a single enemy already built from Lesson 4, GML Visual tab open on obj_spawner's Create event]

"Watch: instead of placing one enemy by hand, I'm going to drag in a Repeat block set to run 5 times, and inside it, a Create Instance block that makes one more enemy at a slightly different position each pass." [Build the Repeat block and the Create Instance block live, run the game] "Five blocks, and I get five independent enemies. If I want ten instead of five, I change one number on the Repeat block, not five copy-pasted objects."

[Demo cue: flip the same event over to its Code tab] "Same event, same five-block idea - here's what it looks like typed instead." [Show the equivalent GML: a plain for loop calling instance_create_layer() five times] "Same logic either way - blocks and typed code are just two different skins on the same idea. GameMaker's the one engine in this class that genuinely lets you flip back and forth on the same event like this, which is exactly why it's demoing loops today - we'll dig into where that flexibility actually helps versus where it doesn't in today's Sidebar."

[Discussion question] "Where else might a loop save you from repeating yourself in a game? " (Examples to draw out if needed: drawing a grid of tiles, giving every enemy in a wave the same starting health, checking every item in an inventory.)

[Timing checkpoint: wrap by 0:25]

[0:25-0:45] Scaling Behavior: Why One Enemy's Code Already Works for Many

"Here's the part that might surprise you. Last week, you wrote the code for what happens when ONE enemy gets hit. Today, without changing a single line of that code, it's going to correctly handle FIVE enemies. Why does that work?" [Pause for guesses]

"Remember the Structure idea from Lesson 4 - that an object's own script should own its own response to being hit? Because you kept that boundary clean, every single copy the loop creates carries its own independent copy of that same behavior. Enemy #3 getting hit has absolutely nothing to do with Enemy #1's code - they're each running their own complete, separate copy."

[Demo cue: same GameMaker project] "Watch what happens if I shoot enemies out of order - the third one, then the first, then the last." [Fire at enemies in a mixed order] "Each one reacts completely independently. Nothing about hitting one affects the others, and I didn't have to write anything special to make that true - it's just a direct result of keeping behavior with the object it belongs to."

"This is a big deal, and it's going to keep paying off all semester. Good structure now means big new features later - like a whole wave of enemies - come almost for free."

[Discussion question] "Can anyone think of a moment in a game where LOTS of things need to react to a similar event independently? " (Examples: every brick in a brick-breaker, every zombie in a horde game, every light in a level flickering on its own.)

[Timing checkpoint: wrap by 0:43-0:45]

[0:45-0:50] A Scratch-Specific Wrinkle: Clones Have No ID

"One quick heads-up, mainly for anyone on Scratch today. Scratch's clones are a little different from instances in every other engine we use - they don't have a unique ID number you can check. That means a Bullet clone literally cannot ask 'which specific Enemy clone am I touching?' - it can only ask 'am I touching any Enemy at all?'"

"So this week, Scratch students are going to move the hit-check from the Bullet's script onto the Enemy's own script instead - each Enemy clone asks 'am I touching a Bullet?' about itself, which works correctly no matter how many enemies exist. It's a real change from last week's code, not just an extension of it - your handout walks through exactly what moves where."

[Timing checkpoint: wrap by 0:50]

[0:50-1:00] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one, and no new assets needed either. Everything today builds directly on your enemy and collision detection from Lesson 4."

"The goal: a row of enemies instead of one, created by a loop. Level 1 is getting that working once. Level 2 is confirming every enemy in the row can be independently destroyed with no

errors. Level 3 is mostly free wins this week - your sound already works, so it's really about adding a bit of visual variety across the row."

[Hand out engine-specific handouts]

"One reminder specific to today: most of you will need to delete the single enemy you manually placed last week before your new loop-spawned row will look right - it's easy to forget and end up with six enemies instead of five."

"Questions before we start lab? ... Let's build an army - well, a small row of them."

Lab Time Notes

- Scratch needs real hands-on support this week - the touching-check has to move from Bullet to Enemy, which is a genuine restructure, not just an addition. Expect this engine's cluster to take longer than usual.
- The single most common bug in every other engine: forgetting to delete last week's manually-placed single enemy, ending up with one extra enemy that doesn't belong to the row.
- Second most common: all spawned enemies stacking on top of each other - almost always means the loop's index variable isn't actually being used in the position calculation.
- Circulate by engine cluster, starting with Scratch this week given the extra complexity there.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout, not given new unrelated work.
- At 1:50, call time and do a 2-minute share-out: ask 2-3 students to show what they built, across different engines if possible.